Study book

Topic 4: Async/await

ProjectHub learning track: React/TypeScript to C#/.NET backend. Week 1, topic 4 of 20.

Brief overview

What this lesson is about. `Task<T>` is C#'s `Promise<T>` and `async`/`await` read the same as in TS. The real difference is **threads**: JS has one thread, .NET has a **thread pool**, so code after an `await` can run on any thread, in parallel with other code. From that follow the key backend ideas: why I/O must be awaited (thread-pool starvation), `CancellationToken` (AbortController), shared state and `lock` (race conditions), and the classic pitfalls (`.Result`, `async void`, fire-and-forget).
What we built. `IProjectService` became async (`...Async`, `Task<T>`, `CancellationToken` on every method). `ProjectService` pretends to be a database (simulated latency that honours cancellation) and protects its shared list with a `lock`. An 8-section demo measures everything; a separate experiment proves what happens without the lock.
Checkpoint. Explain why backend I/O should be awaited; pass a `CancellationToken` through one path (done: every service method).

Contents

Part	What is inside
1	TS to C# mapping (Promise, Promise.all, AbortController, ...)
2	One thread vs a thread pool: what really happens at an await
3	Why backend I/O must be awaited (the checkpoint) and what async is not
4	Writing async methods: return types, state machine, exceptions in tasks
5	CancellationToken
6	Shared state and lock: race conditions, check-then-act, your three questions
7	Pitfalls (interview favourites)
8	Our code explained: IProjectService and ProjectService
9	The demo and the race experiment, with real output
10	Debugging in Rider: stepping over awaits, threads, fire-and-forget
11	Your answers, corrections, interview Q&A
12	Summary, cheat sheet, checklist, what comes next

Part 1. The mapping you already know

TypeScript / JS	C#	Note
`Promise<T>`	`Task<T>`	a value available later
`Promise<void>`	`Task`	no result, just done (or failed)
`async function f(): Promise<T>`	`async Task<T> FAsync()`	name ends with **Async** by convention
`await p`	`await t`	unwraps the result or re-throws the error
`Promise.all([a, b])`	`Task.WhenAll(a, b)`	wait for all; results in **input order**
`Promise.race([a, b])`	`Task.WhenAny(a, b)`	first to finish
`Promise.resolve(x)`	`Task.FromResult(x)`	already completed
`new Promise(r => setTimeout(r, 300))`	`Task.Delay(300)`	wait without blocking
`AbortController`	`CancellationTokenSource`	the side that **cancels**
`AbortSignal`	`CancellationToken`	the side you **pass around**
rejected promise	faulted task	exception stored until awaited

Part 2. One thread vs a thread pool

- **JS / Node:** one thread + event loop. After an `await` your code continues on the same single thread (via the microtask queue); two pieces of your code never run at the same moment.
- **.NET:** a **thread pool** (starts with about one thread per CPU core, grows slowly). An ASP.NET request runs on a pool thread. At an `await` on real I/O: (1) the method **pauses** and its thread **goes back to the pool** to serve others; (2) when the I/O completes, **any free pool thread** continues the method.

Demo section 3: thread ids across 8 awaits: `7 -> 7 -> 8 -> 8 -> 8 -> 8 -> 7 -> 8 -> 8`.

Common mistake

You first answered 'after await runs, the task needs a lock'. Correction: `await` itself needs no lock. Because continuations run on **any** pool thread, **possibly in parallel**, code that touches **shared mutable data** (our list) needs a lock. That is the consequence, not a rule of await.

Part 3. Why backend I/O must be awaited (the checkpoint)

A server handles many requests with a **limited number of threads**, and most of a request's time is **waiting** (database, other APIs, files).

	Blocking wait (`Thread.Sleep`, `.Result`, `.Wait()`, sync DB calls)	`await`
During the wait	the thread sits idle and cannot serve anyone	the thread goes back to the pool and serves other requests
Under load	all threads stuck -> **thread-pool starvation** -> requests queue -> latency, timeouts	a few threads serve thousands of mostly-waiting requests

```
50 requests, each waiting 500 ms, 4-core machine (demo section 4):
Blocking  (Thread.Sleep):        5511 ms   <- each request HOLDS a thread; the pool grows slowly
Async     (await Task.Delay):     501 ms   <- no thread held while waiting
```

Checkpoint answer

Model answer: 'While a request awaits I/O, its thread goes back to the pool and serves other requests. Blocking would keep the thread idle; with many concurrent requests the pool runs out (**thread-pool starvation**), so requests queue and time out. Awaiting lets a few threads serve many requests.'
Order of cause and effect: starvation (all threads stuck) comes **first**; queueing and timeouts follow.

What async is NOT

- It does **not** make one request faster: a 300 ms query still takes 300 ms. It improves **throughput / scalability**.
- It does **not** help **CPU-bound** work (heavy calculations): there is no waiting to give back.
- **Async all the way:** once a method is async, callers await it up to the controller. A `.Result` anywhere brings blocking back.

Part 4. Writing async methods

```
public async Task<Project?> GetByIdAsync(int id, CancellationToken cancellationToken = default)
{
    await SimulateDatabaseCallAsync(cancellationToken); // pause here, thread returns to the pool
    lock (_gate)
    {
        return FindById(id);   // return a Project?; the compiler wraps it in Task<Project?>
    }
}
```

- `async` allows `await` inside and makes the compiler **wrap the return value** in a Task (write `return project;`, not `Task.FromResult`).
- Under the hood: a compiler-generated **state machine** that pauses and resumes (like TS generators).
- Return types: `Task<T>`, `Task`, `ValueTask<T>` (hot-path optimization; recognize it), `async void` (**only** event handlers).
- A method **without** `async` can still return a Task: `LoggingProjectService.GetAllAsync` just returns `inner.GetAllAsync(...)` (pass-through).

Exceptions travel inside the task

An exception inside an async method is stored in the Task (status `Faulted`) and **re-thrown at the** `await`: `try { await x; } catch` works like JS. A task that is never awaited loses its exception silently: the C# version of an unhandled promise rejection. Demo section 6: `Status = WaitingForActivation` right after the call, then the exception at the await, status `Faulted`.

Part 5. CancellationToken

Why a backend needs it: the user closes the tab or the client times out, but the server keeps running a long query for nobody. ASP.NET creates a token per request that is **cancelled when the client disconnects** (`HttpContext.RequestAborted`; in a controller just add a `CancellationToken` parameter, topic 6). Passed down to EF Core, the database query stops too.

```
using var cts = new CancellationTokenSource();       // TS: const ctrl = new AbortController()
cts.CancelAfter(TimeSpan.FromMilliseconds(100));     // TS: setTimeout(() => ctrl.abort(), 100)
await service.GetAllAsync(cts.Token);                 // TS: fetch(url, { signal: ctrl.signal })
// -> OperationCanceledException (TaskCanceledException is a subclass)
```

- The **caller** owns the `CancellationTokenSource` and decides when to cancel; methods **receive** the token and **pass it down** to everything they await.
- Convention: last parameter, named `cancellationToken`, often `= default` (= `CancellationToken.None`, never cancelled).
- **Cooperative**: nothing is killed; code checks the token (`Task.Delay`, EF Core, `HttpClient` do it; manual: `ThrowIfCancellationRequested()`).
- Catch `OperationCanceledException` (the base). Cancellation is not a failure to log as an error.
- Demo section 5: a 300 ms call cancelled after ~110 ms; `CreateAsync` with an already-cancelled token adds nothing.

Part 6. Shared state and lock

```
private readonly Lock _gate = new();   // .NET 9+ lock type (older code: private readonly object _gate = new();)

lock (_gate)                           // only ONE thread at a time may run this block
{
    if (_projects.Any(p => ...same name...)) throw ...;   // check
    _projects.Add(project);                                // act
}
```

- `List<T>` is **not thread-safe**, and `_nextProjectId++` is not atomic (two threads can read the same value -> duplicate ids).
- **Check-then-act race:** if 'is the name taken?' and 'add it' are separate steps, two concurrent creates can both pass the check.
- **No `await` inside a `lock`** (compile error): see 6.3.
- With EF Core (topic 10) the **database** handles concurrency (transactions, unique index, identity ids); a `lock` only protects inside one process.

6.1 Your question: how does the lock give 'exactly 1 created, 4 rejected'?

Your scenario: 'thread 1 passes the check and starts adding; thread 2 also passes because thread 1 is not done'. That is exactly what happens **without** the lock. `lock` is a door with one key: a second thread reaching `lock (_gate)` **waits there, before the check**, until the first thread leaves the block.

Time	Without lock: thread 1	Without lock: thread 2	With lock: thread 1	With lock: thread 2
t1	check: taken? No		enters lock	
t2		check: taken? No	check: taken? No	reaches lock -> **waits**
t3	Add -> created		Add -> created	waiting
t4		Add -> **duplicate**	leaves the block	
t5				enters lock
t6				check: taken? **Yes** -> rejected

The check and the add become one **atomic** unit. Separate locks for check and add would reopen the gap. JS never needs this: one thread, so synchronous code is never interrupted. Terms: **race condition** (result depends on which thread wins), **critical section** (block only one thread may run), **atomic**.

The experiment: the same logic without and with a lock

docs/studybooks/notes/04-race-experiment.cs (Thread.Sleep(10) widens the gap so the race is easy to see)

```
// Same check-then-act logic, once WITHOUT and once WITH a lock.
// Thread.Sleep(10) between check and add widens the gap so the race is easy to see.

Console.WriteLine($"Without lock: {Run(useLock: false)}");
Console.WriteLine($"With lock:    {Run(useLock: true)}");

static string Run(bool useLock)
{
    var names = new List<string>();
    var gate = new Lock();
    var results = new string[5];

    void Create(int i)
    {
        if (useLock)
        {
            lock (gate) { results[i] = CheckThenAdd(); }
        }
        else
        {
            results[i] = CheckThenAdd();
        }
    }

    string CheckThenAdd()
    {
        if (names.Contains("Race project")) return "rejected"; // check
        Thread.Sleep(10);                                      // the gap
        names.Add("Race project");                             // act
        return "created";
    }

    Parallel.For(0, 5, Create); // 5 threads at the same time
    return $"{string.Join(", ", results)}  -> {names.Count} project(s) named 'Race project'";
}
```

Output when the study book was generated:

```
Without lock: created, created, created, created, created  -> 4 project(s) named 'Race
    project'
With lock:    created, rejected, rejected, rejected, rejected  -> 1 project(s) named 'Race
    project'
```

Even worse than duplicates

Look at the 'without lock' line: 5 creates all reported 'created', but the list may hold **fewer** than 5 entries (one run gave 4). Two threads called `List.Add` at the same moment and one item was **silently lost**. Without a lock, a `List<T>` does not only allow duplicates: it can lose or corrupt data. Race bugs are rare, random and load-dependent: 'it worked when I tested it' proves nothing.

6.2 Your conclusion: slow wait outside, fast work inside, one lock

You concluded: 'we get the data in the await, then inside the lock we only do fast C# things, so threads do not wait long; it is like one single lock for all threads'. Correct, with two precisions:

1. **Keep critical sections short.** The 300 ms wait is **outside** the lock; inside are microseconds, so waiting threads wait microseconds. With the wait inside, 5 creates would take ~1.5 s one after another. In our fake the await only *simulates* the database time; the data is read from the list inside the lock. With a real database the data comes through the await and the DB handles concurrency.

2. **One lock per `ProjectService` object** (`_gate` is an instance field). Threads using the same service object share the lock (they share the list). Different objects = different lists + different locks. Topic 7 registers the service as a **singleton**: one object, one list, one lock for all requests.

Design trade-off: one lock for every operation is **coarse-grained**: the simplest correct choice; fine because the work inside is tiny. Finer tools exist (`ConcurrentDictionary`, reader/writer locks).

6.3 Your question: 'why were all 5 rejected?'

You saw `rejected, rejected, rejected, rejected, created` and expected the **first** call to win. The line actually has **1 created** (the last). Four runs gave: last, last, first, last.

1. `Task.WhenAll` returns results in **input order** (like `Promise.all`), not completion order: position 5 = 'the 5th call started'.
2. **Started first does not mean enters the lock first.** The five 300 ms waits end almost together; the **thread scheduler** decides who takes the lock.

> **Remember**
> A lock guarantees **exactly one winner**, not **which one**. Correct code must not depend on the winner; if order matters, use explicit ordering (a DB sequence, a queue). If the title was changed to an **existing** name (e.g. 'Mobile app'), all 5 rejected is correct.

6.4 Why await is forbidden inside a lock

A lock **belongs to the thread that took it**. After an `await` the method may continue on **another thread**, which would try to release a lock it does not own. So the compiler forbids `await` inside `lock`. ('Locks are for short operations' is the design guideline, not the reason for this rule.)

Part 7. Pitfalls (interview favourites)

Pitfall	What goes wrong	Do instead
`.Result`, `.Wait()`, `.GetAwaiter().GetResult()` (sync over async)	blocks the thread (starvation); in UI apps / old ASP.NET it can **deadlock**	`await` all the way
`async void`	cannot be awaited; an exception inside **crashes the process**	`async Task` (event handlers only)
async call without `await` (fire-and-forget)	errors silently lost; work may outlive the request; warning **CS4014**	`await` it (or a real background job system)
sequential awaits for independent calls	slower (demo 2: 902 ms vs 301 ms)	start all, then `await Task.WhenAll(...)`
`Thread.Sleep` in async code	blocks a thread	`await Task.Delay(...)`
not passing the `CancellationToken` down	work continues after the client left	pass it to every async call
`Task.Run` around I/O in ASP.NET	just moves the blocking to another pool thread	a real async API
`Task.WhenAll` over queries on one EF `DbContext`	throws: a DbContext does not support parallel queries	sequential awaits, or separate contexts

`ConfigureAwait(false)`: seen in library code ('do not resume on the original synchronization context'). ASP.NET Core has no synchronization context, so application code does not need it.

Part 8. Our code, explained

8.1 The async contract

src/ProjectHub.Api/Services/IProjectService.cs

```
using ProjectHub.Api.Entities;

namespace ProjectHub.Api.Services;

// Async because the real implementation (EF Core + SQL Server, Topic 10) does I/O.
// Every method accepts a CancellationToken so a caller can stop work that is no longer needed.
public interface IProjectService
{
    Task<IReadOnlyList<Project>> GetAllAsync(CancellationToken cancellationToken = default);

    // Returns null when the project does not exist: "not found" is a normal answer here.
    Task<Project?> GetByIdAsync(int id, CancellationToken cancellationToken = default);

    // Throws BusinessRuleException when the name is empty, too long, or already used.
    Task<Project> CreateAsync(string name, string? description, CancellationToken cancellationToken = default);

    // Throws NotFoundException when the project does not exist.
    Task<TaskItem> AddTaskAsync(int projectId, string title, CancellationToken cancellationToken = default);
}
```

Why async when the data is an in-memory list? The interface describes the *real* system. EF Core (topic 10) does I/O and must be async; if the interface stayed synchronous, every caller (controllers, tests) would change in topic 10. Now only the implementation changes: the 'swap implementations' benefit from topic 3.

8.2 The implementation

src/ProjectHub.Api/Services/ProjectService.cs

```
using ProjectHub.Api.Entities;
using ProjectHub.Api.Exceptions;

namespace ProjectHub.Api.Services;

// In-memory implementation. Topic 10 replaces the List with EF Core + SQL Server.
// It pretends to be a database: each call can wait for a simulated "round-trip" (simulatedLatency).
public class ProjectService : IProjectService
{
    public const int MaxNameLength = 100;

    private readonly List<Project> _projects;
    private readonly TimeSpan _simulatedLatency;

    // Async callers can run at the same time on different threads, so every access to the
    // shared list and id counters happens inside lock (_gate): one thread at a time.
    private readonly Lock _gate = new();
    private int _nextProjectId;
    private int _nextTaskId;

    public ProjectService(IEnumerable<Project> initialProjects, TimeSpan simulatedLatency = default)
    {
        _projects = initialProjects.ToList();
        _simulatedLatency = simulatedLatency;

        // DefaultIfEmpty(0): Max() on an empty sequence would throw (Topic 2 gotcha).
        _nextProjectId = _projects.Select(p => p.Id).DefaultIfEmpty(0).Max() + 1;
        _nextTaskId = _projects.SelectMany(p => p.Tasks).Select(t => t.Id).DefaultIfEmpty(0).Max() + 1;
    }

    public async Task<IReadOnlyList<Project>> GetAllAsync(CancellationToken cancellationToken = default)
    {
        await SimulateDatabaseCallAsync(cancellationToken);

        lock (_gate)
        {
            // A snapshot copy: safe to loop over even if another caller adds a project meanwhile.
            return _projects.ToList().AsReadOnly();
        }
    }

    public async Task<Project?> GetByIdAsync(int id, CancellationToken cancellationToken = default)
    {
        await SimulateDatabaseCallAsync(cancellationToken);

        lock (_gate)
        {
            return FindById(id);
        }
    }

    public async Task<Project> CreateAsync(string name, string? description, CancellationToken cancellationToken = default)
    {
        // Validate the input first: no point "calling the database" with a bad request.
        var trimmedName = name.Trim();
```

(continued)

```
        if (trimmedName.Length == 0)
        {
            throw new BusinessRuleException("Project name is required.");
        }

        if (trimmedName.Length > MaxNameLength)
        {
            throw new BusinessRuleException($"Project name must be at most {MaxNameLength} characters.");
        }

        await SimulateDatabaseCallAsync(cancellationToken);

        lock (_gate)
        {
            // Check AND add inside the same lock: otherwise two concurrent calls with the same name
            // could both pass the check before either one adds (a "check-then-act" race).
            if (_projects.Any(p => string.Equals(p.Name, trimmedName, StringComparison.OrdinalIgnoreCase)))
            {
                throw new BusinessRuleException($"A project named '{trimmedName}' already exists.");
            }

            var project = new Project
            {
                Id = _nextProjectId++,
                Name = trimmedName,
                Description = description,
            };
            _projects.Add(project);
            return project;
        }
    }

    public async Task<TaskItem> AddTaskAsync(int projectId, string title, CancellationToken cancellationToken = default)
    {
        var trimmedTitle = title.Trim();
        if (trimmedTitle.Length == 0)
        {
            throw new BusinessRuleException("Task title is required.");
        }

        await SimulateDatabaseCallAsync(cancellationToken);

        lock (_gate)
        {
            var project = FindById(projectId) ?? throw new NotFoundException(nameof(Project), projectId);

            var task = new TaskItem
            {
                Id = _nextTaskId++,
                ProjectId = project.Id,
                Title = trimmedTitle,
            };
            project.Tasks.Add(task);
            return task;
```

(continued)

```
        }
    }

    // Call only while holding _gate.
    private Project? FindById(int id) => _projects.SingleOrDefault(p => p.Id == id);

    // Stands in for the network + database time of a real query. Task.Delay honours the token:
    // if the caller cancels, it stops waiting and throws OperationCanceledException.
    private Task SimulateDatabaseCallAsync(CancellationToken cancellationToken) =>
        Task.Delay(_simulatedLatency, cancellationToken);
}
```

Change	Why
constructor parameter `TimeSpan simulatedLatency = default`	pretends to be a database; `default` = zero so tests run instantly; the demo uses 300 ms
`SimulateDatabaseCallAsync => Task.Delay(_simulatedLatency, cancellationToken)`	stands in for network + SQL time; honours the token (an already-cancelled token throws even for zero delay); not `async`, it just returns the task

Change	Why
`public async Task<...> XxxAsync(..., CancellationToken cancellationToken = default)`	async methods; the token is passed down
validation **before** the await	do not 'call the database' with an invalid request. Side effect: `AddTaskAsync(42, "")` now reports the empty title first (the topic 3 demo was adjusted)
`private readonly Lock _gate = new();` + `lock (_gate)` around list and counters	continuations run in parallel on different threads
name check and `Add` in **one** lock	prevents the check-then-act race
`GetAllAsync` returns `_projects.ToList().AsReadOnly()`	a **snapshot**: callers can loop while others add (no 'Collection was modified'); still read-only; entities still shared (topic 3 leak, fixed by DTOs in topic 6)
`private Project? FindById(int id)` without a lock	helper that assumes the caller holds the lock

Part 9. The demo: playground/04-async.cs

Run from the repo root: `dotnet run playground/04-async.cs` (listed in Rider's `playground` solution folder). Timings depend on the machine.

#	Shows
1	`Task<T>` pending right after the call; `await` unwraps it
2	sequential awaits (~900 ms) vs `Task.WhenAll` (~300 ms)
3	thread id changes across awaits
4	50 blocking vs 50 awaited waits: thread-pool starvation, measured
5	cancellation with a timeout; an already-cancelled token changes nothing
6	exception stored in the task (Faulted), thrown at the await
7	concurrent creates: the lock keeps names unique and ids distinct
8	`.Result` blocks the thread for the whole wait

Full output (captured in the cloud container, 4 cores):

```
=== 1. Task<T> is a Promise<T>; await unwraps it ===
Right after the call: IsCompleted = False   (still 'pending')
After await:          IsCompleted = True, project = Website redesign

=== 2. Sequential awaits vs Task.WhenAll (Promise.all) ===
Sequential:   902 ms  (3 x 300 ms, one after another)
WhenAll:      301 ms  (the three waits overlap)  -> Website redesign, Mobile app, Internal
    tools

=== 3. Threads: the code after an await may continue on ANOTHER thread ===
Thread id before the first await, then after each of 8 awaits: 5 -> 5 -> 8 -> 5 -> 8 -> 8 ->
    8 -> 5 -> 8
Same or different ids: any free thread-pool thread can continue the method (JS always has
    one thread).

=== 4. Why backend I/O must be awaited: blocking vs non-blocking ===
Blocking  (Thread.Sleep):        5510 ms  (each request HOLDS a thread while waiting)
Async     (await Task.Delay):     501 ms  (no thread is held while waiting)
Processor count: 4  (the pool starts with about this many threads and grows slowly)

=== 5. CancellationToken: stop work nobody needs any more (AbortController) ===
Cancelled after 109 ms: TaskCanceledException
Create with a cancelled token -> OperationCanceledException; project exists? False

=== 6. Exceptions travel inside the Task until you await it ===
Called CreateAsync without await: no exception yet. Status = WaitingForActivation
Thrown at the await: A project named 'Mobile app' already exists.   (Status was Faulted)

=== 7. Shared state + concurrency: why ProjectService uses lock ===
5 concurrent creates with the SAME name: rejected, rejected, rejected, rejected, created
20 concurrent creates: 20 projects, 20 distinct ids

=== 8. Pitfall: .Result blocks the thread (sync-over-async) ===
.Result returned 'Mobile app' after 300 ms, with the thread blocked the whole time
In ASP.NET Core: use 'await' instead. .Result/.Wait() waste threads (and deadlock in some
    older app types).
```

Part 10. Debugging: awaits and threads in Rider

Temporary snippet in Program.cs before app.Run() (top-level Program.cs can await directly):

```
var asyncService = new ProjectHub.Api.Services.ProjectService(
    ProjectHub.Api.Data.SampleData.CreateProjects(), TimeSpan.FromMilliseconds(200));

Console.WriteLine($"Before: thread {Environment.CurrentManagedThreadId}");
var first = await asyncService.GetByIdAsync(1);                 // BREAKPOINT A
Console.WriteLine($"After:  thread {Environment.CurrentManagedThreadId}");

var three = await Task.WhenAll(
    asyncService.GetByIdAsync(1), asyncService.GetByIdAsync(2), asyncService.GetByIdAsync(3));
Console.WriteLine($"Got {three.Length} projects");
```

1. Breakpoint **A** on `var first = await ...;` breakpoint **B** in `ProjectService.GetByIdAsync` on `return FindById(id);`.
2. Debug; at A use **Step Over**: Rider steps over the await (the wait happens, you land on the next line).
3. Resume: B is hit for each `WhenAll` call; look at `id`, the Threads list and the **Parallel Stacks** tab.
4. Break it: add `asyncService.CreateAsync("Mobile app", null);` (no await) + `Console.WriteLine("Continued happily");`; then add `await`.
5. Clean up the snippet and breakpoints.

What you observed

Step	Result	Meaning
Step Over at A	B was hit with `id = 1`	breakpoints inside the awaited method **are** hit while stepping over an await
WhenAll	B hit with `id` 3, 2, 1	reverse of the start order: started first != continues first (same as demo 7)
Without `await`	the debugger showed the `BusinessRuleException` being thrown, but 'Continued happily' was printed	the error was **silently lost** inside a faulted task nobody awaited (fire-and-forget, CS4014)
With `await`	no 'Continued happily'	the exception surfaced at the await and stopped the program

Part 11. Your answers, corrections, interview Q&A

11.1 CHECK round 1

Question	Your answer	Complete answer
1. Task vs Promise	same: a result later; JS continues via event loop/microtasks (good); 'after await the task needs a lock' (**wrong**)	continuation on any pool thread, possibly in parallel -> shared data needs a lock; await itself needs none
2. Why await I/O (checkpoint)	thread goes back to the pool while the DB works (core right, not crisp)	see the model answer in Part 3 (starvation, queueing, few threads serve many)
3. Faster?	'no' (right); 'when we don't need await, async makes no sense'	improves throughput/scalability, not single-request speed; no help for CPU-bound work
4. CancellationToken	cancels the operation, EF stops; the requester creates it (right)	plus: methods pass it down; ASP.NET creates it per request (cancelled on client disconnect)
5. .Result / .Wait()	blocks the thread until the end (right)	plus: thread-pool starvation; deadlock in UI / old ASP.NET
6. async void / no await	'cannot be awaited' (half)	exception in async void crashes the process; fire-and-forget loses errors (you saw it in debug)
7. lock + same lock	right	
8. No await in lock	'lock is for superfast operations' (**wrong reason**)	lock belongs to a thread; after await another thread may continue -> compile error
9. Async interface	for the future real DB/HTTP (right)	
10. WhenAll vs sequential	WhenAll for independent calls; sequential when dependent (right)	plus: no parallel queries on one EF DbContext

11.2 Re-check

Question	Your answer	Verdict
A. Checkpoint	thread goes back to the pool during async work; otherwise it sits idle, requests queue, starvation	right; order: starvation first, then queueing
B. await in lock	lock belongs to the thread that took it; after await another thread may continue, release impossible	right (it is a compile error)
C. What async improves / not	threads are constantly used, not idle	half: say **throughput/scalability**, and **no help for CPU-bound work**

11.3 Interview questions with model answers

Q: Task vs Promise?

Both represent a future result; await unwraps or re-throws. In .NET the continuation runs on any thread-pool thread, possibly in parallel; JS has one thread.

Q: Why should backend I/O be awaited?

The thread returns to the pool while waiting and serves other requests; blocking keeps threads idle and leads to thread-pool starvation, queueing and timeouts.

Q: Does async make a request faster?

No. It improves throughput/scalability. It does not help CPU-bound work.

Q: What is thread-pool starvation?

All pool threads are blocked waiting, so new work queues up; the pool grows slowly; latency and timeouts follow.

Q: What is a CancellationToken? Who creates it?

A signal that work is no longer needed (AbortSignal). The caller owns the CancellationTokenSource; methods receive the token and pass it down. ASP.NET supplies one per request, cancelled when the client disconnects.

Q: Why is .Result / .Wait() bad?

Sync over async: blocks a thread (starvation) and can deadlock where a synchronization context exists (UI, old ASP.NET).

Q: What is wrong with async void?

It cannot be awaited and an exception inside it crashes the process. Use async Task; async void only for event handlers.

Q: What happens if you call an async method without await?

Fire-and-forget: the exception is stored in a task nobody observes and is lost; the work may outlive the request (CS4014).

Q: Where does an exception in an async method surface?

It is stored in the task (Faulted) and re-thrown at the await.

Q: Task.WhenAll vs sequential awaits?

WhenAll for independent operations (they overlap); sequential when each depends on the previous. Not for parallel queries on one EF DbContext.

Q: What order does Task.WhenAll return results in?

The order of the input tasks, not completion order (like Promise.all).

Q: What is a race condition? Check-then-act?

The result depends on thread timing. Check-then-act: two threads both pass a check before either acts; fix by making check+act atomic (one lock, or a DB constraint).

Q: Why can't you await inside a lock?

A lock belongs to the thread that took it; after await another thread may continue and could not release it. Compile error.

Q: Is List<T> thread-safe?

No: concurrent Add can lose items or corrupt the list; protect it with a lock or use a concurrent collection.

Q: What does a lock guarantee about the winner?

Exactly one thread at a time in the critical section; not which thread goes first.

Q: Why keep critical sections short?

Every other thread waits while one holds the lock; never do I/O inside it.

Q: Why make the interface async if the implementation is in-memory?

The interface describes the real system (EF Core I/O). Making it async now means callers will not change when the implementation does.

Q: What does ConfigureAwait(false) do? Needed in ASP.NET Core?

Do not resume on the captured synchronization context. ASP.NET Core has none, so application code does not need it.

Part 12. Summary

Key takeaways

`Task<T>` = `Promise<T>`; `Task.WhenAll` = `Promise.all` (input order); `CancellationToken` = `AbortSignal`.
Thread pool: at an await on I/O the thread returns to the pool; any free thread continues the method, possibly in parallel with other code.
Why await I/O: threads serve other requests while waiting; blocking -> **thread-pool starvation** -> queueing, timeouts. Async improves **throughput**, not single-request speed; no help for CPU-bound work.
Async methods: `async Task<T> XxxAsync(..., CancellationToken cancellationToken = default)`; exceptions surface at the await; async all the way.
Cancellation: caller owns the source; methods pass the token down; ASP.NET cancels it when the client disconnects; cooperative; not an error.
Shared state: `List<T>` is not thread-safe; check + act in **one** `lock`; keep critical sections short; no await inside a lock; one lock per service object; a lock guarantees one winner, not which.
Pitfalls: `.Result`/`.Wait()`, `async void`, fire-and-forget, sequential independent awaits, `Thread.Sleep`, parallel queries on one DbContext.

Cheat sheet

TS / JS	C#
`async function getById(id): Promise<Project | null>`	`async Task<Project?> GetByIdAsync(int id, CancellationToken ct = default)`
`await Promise.all([a(), b()])`	`await Task.WhenAll(A(), B())`
`await new Promise(r => setTimeout(r, 300))`	`await Task.Delay(300, ct)`
`const c = new AbortController(); c.abort()`	`using var cts = new CancellationTokenSource(); cts.Cancel()`
`fetch(url, { signal })`	`await http.GetAsync(url, ct)`
`try { await p } catch (e) { }`	`try { await t; } catch (SomeException ex) { }`
(no equivalent: one thread)	`lock (_gate) { /* check + act */ }`

Checklist: can you do this without notes?

- Give the checkpoint answer in two sentences with 'thread', 'pool', 'starvation'.
- Explain what happens to the thread at an await, and why code after it may run on another thread.
- Explain what async improves and where it does not help.
- Explain CancellationToken: who creates it, who passes it, where ASP.NET gets it.
- Name the pitfalls: .Result, async void, fire-and-forget, Thread.Sleep, WhenAll on one DbContext.
- Draw the two timelines (with / without lock) for check-then-act.
- Explain why await is not allowed inside a lock.
- Explain why IProjectService is async although the data is in memory.

What comes next: Topic 5 (Debugging fundamentals, Debug Lab A)

A focused local lab in Rider: find a real bug without `Console.WriteLine`, using breakpoints, Step Into/Over/Out, Watches, conditional breakpoints, the call stack and exception settings. You have already used most of these tools; topic 5 puts them together on a planted bug in ProjectHub.